

Link to doc: <https://goo.gl/qLiZLA>

AMOS Project 1 Zero Downtime Linux Updates

Instructions

The project report will be published on our blog. It should be short and sweet, focussed on what you achieved.

“I did not have time to write a short letter, so I wrote a long letter instead.” Attributed to Mark Twain

Being concise is hard work and takes time. Please write as professional a text as you can. Use formal language and correct grammar.

For illustrations please use a persona rather than “test X” or “testperson12”.

Prior examples (not necessarily following our instructions, sadly), can be found here:

<https://dirkriehle.com/2021/03/02/summary-of-the-winter-2020-21-amos-projects/>

<https://oss.cs.fau.de/2018/08/01/show-casing-the-2017-amos-project-simulating-a-cars-ecus-using-a-raspberry-pi-5/>

Template

Please use the following template for creating your project report.

Semester	Summer 2026
Project name	Zero Downtime Linux Updates
Project mission	To design and develop a secure, Rust-based edge orchestrator for Weber's Edge IPC that enables near zero-downtime updates for both the operating system and application layers. Our mission is to implement a management agent that leverages Fedora IoT, bootc, and Podman to ensure system updates and reliability. We create a cloud containing a database, which manages edge device information, and the running and desired OS and application versions. Users should be able to create, read, update, and delete devices, device groups, OS, and application information.
Industry partner	Weber Food Technology SE & Co. KG

Team logo



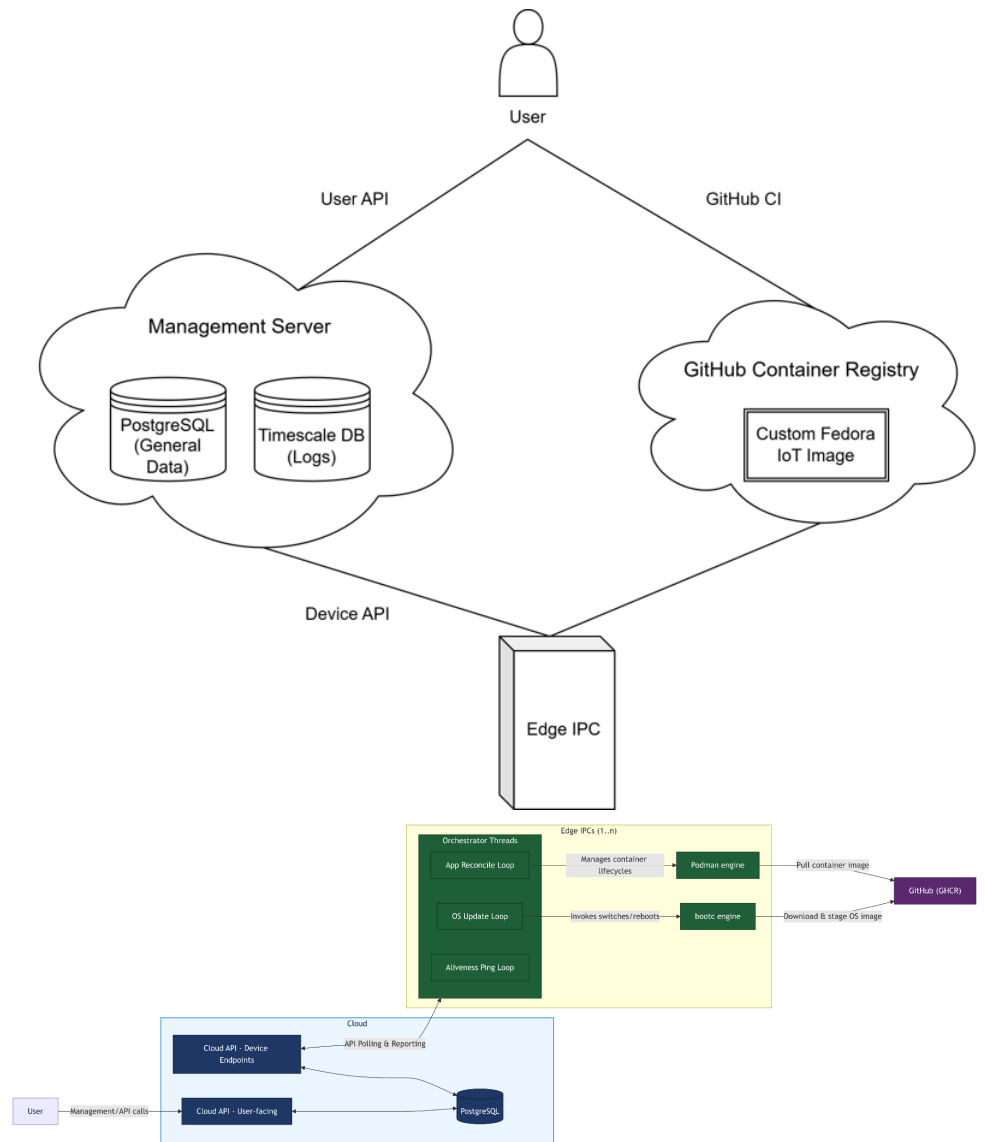
Project summary

An update system for fleets of Linux edge devices (industrial IPCs) where the entire operating system ships as a bootable container image. Devices are provisioned once, then updated forever by pulling a new image — each OS update is atomic and staged, and the OS rolls back automatically if a new version fails to boot or fails its health checks.

How it works

- The OS is a container image. We build a bootable OS image with bootc and publish it to GHCR, the same way you'd build and push an application container.
- An on-device agent drives updates. amos-orchestrator runs on each device, polls the cloud API for the OS and application versions it should be running, and invokes bootc and podman to converge onto that state.
- The cloud holds the desired state. A user manages the fleet through the cloud API; the cloud persists per-device state in PostgreSQL and hands each device its target state on request.

Project illustration



Team photo

/

Project repository

<https://github.com/amosproj/amos2026ss01-zero-downtime-linux-updates>

LinkedIn handles

m0ritz